

CovR: Coverage-Aware Hardware Verification via Reasoning-Guided Reinforcement Learning

Manar Abdelatty, Maryam Nouh, Sherief Reda

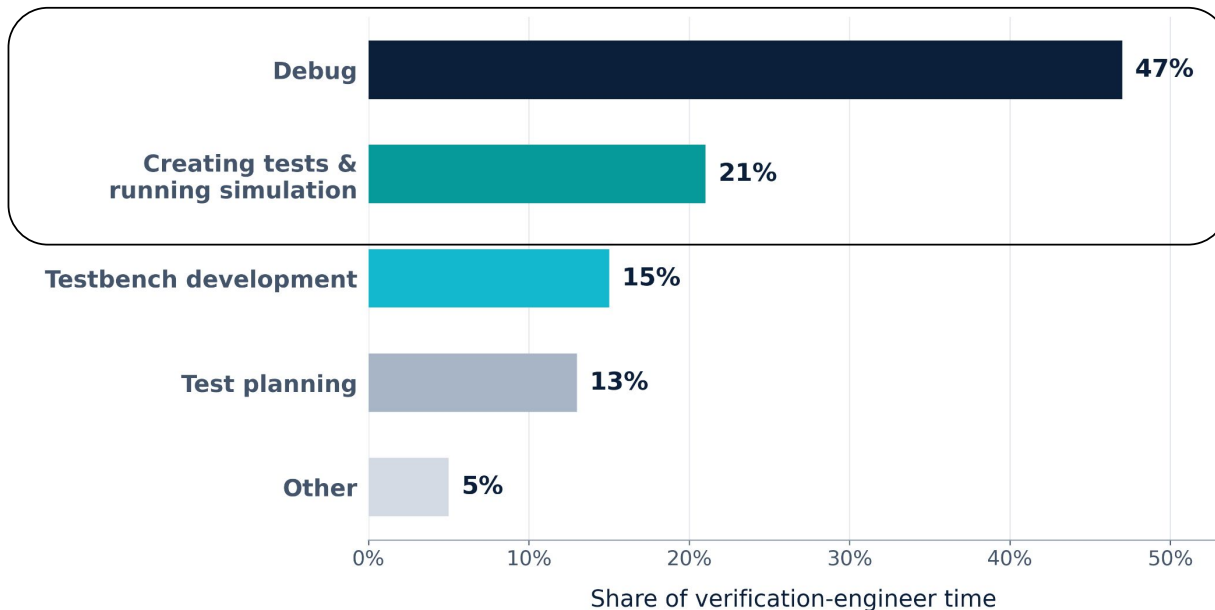
Brown University, Providence, RI



BROWN

Where Verification Engineers Spend their Effort?

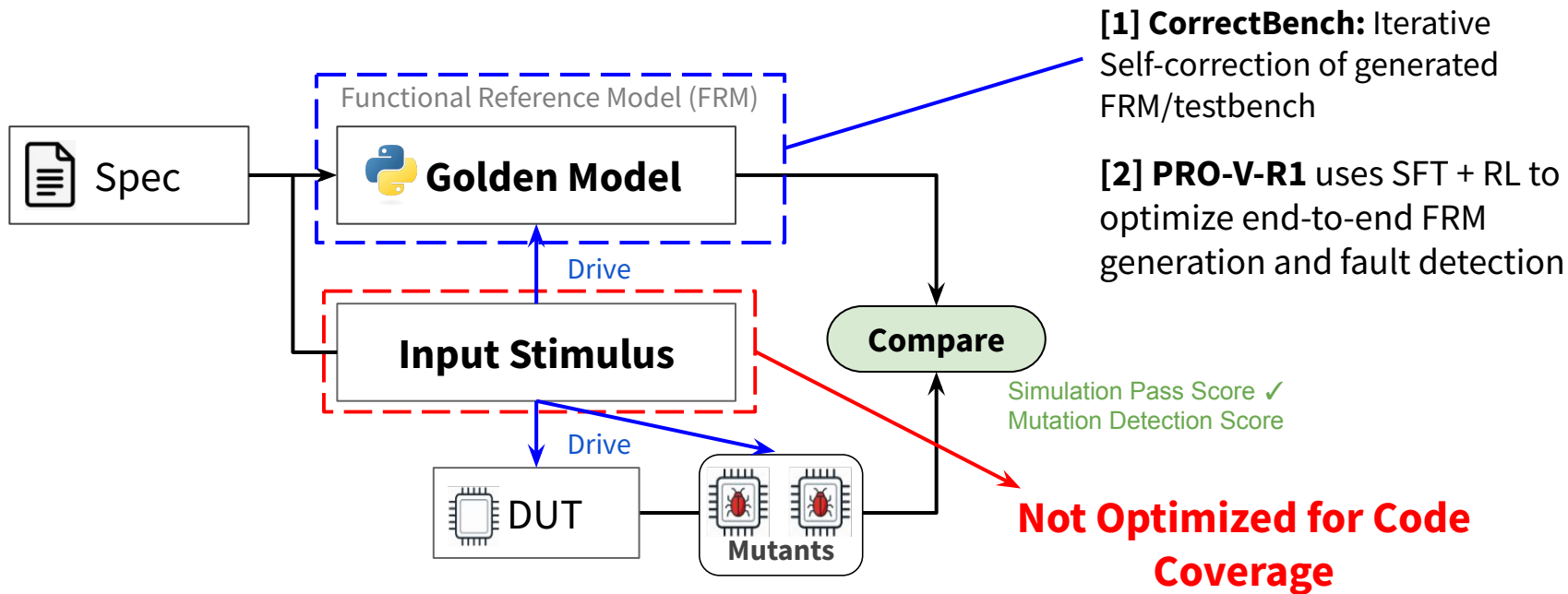
- Approximately **55–56%** of IC/ASIC project time is spent on verification



Source: Siemens EDA/Wilson Research Group Functional Verification Study, 2022.

LLM-Assisted Hardware Verification Workflows

- Current LLM-based full verification workflows **overlook test stimulus coverage**.

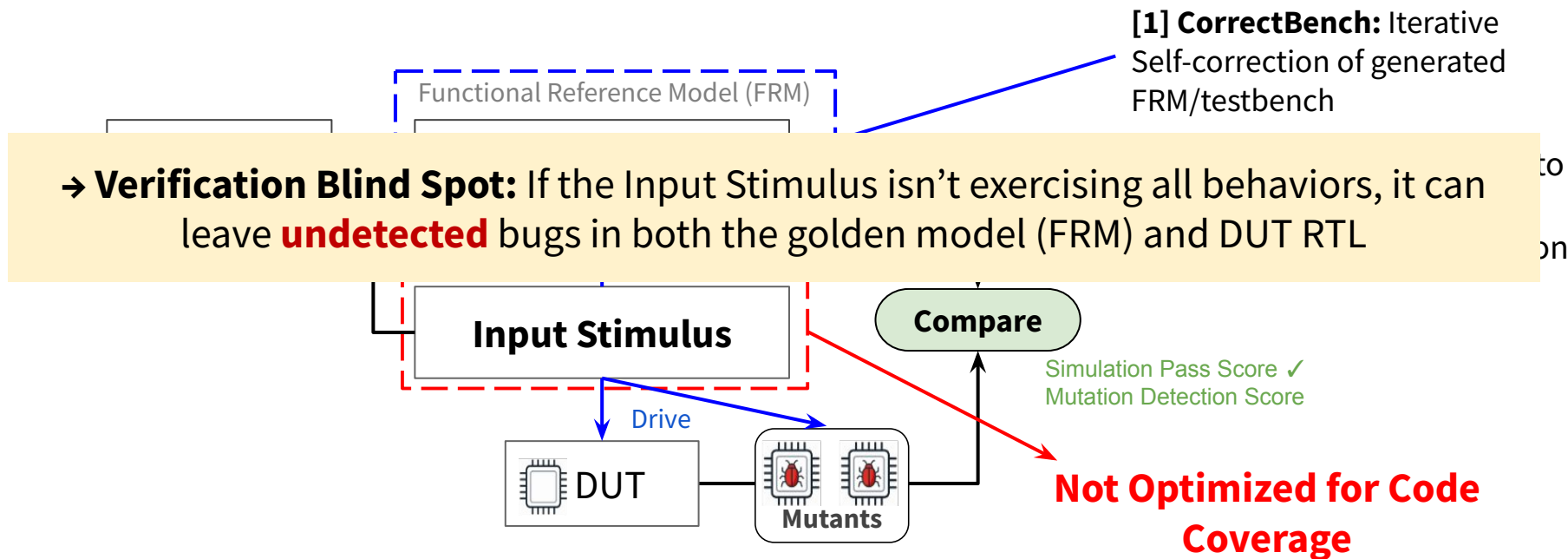


[1] Qiu, Ruidi, et al. "Correctbench: Automatic testbench generation with functional self-correction using llms for hdl design." *DATE*, 2025.

[2] Zhao, Yujie, et al. "PRO-V-R1: Reasoning enhanced programming agent for RTL verification." *DAC* (2026).

LLM-Assisted Hardware Verification Workflows

- Current LLM-based full verification workflows **overlook test stimulus coverage**.



[1] Qiu, Ruidi, et al. "Correctbench: Automatic testbench generation with functional self-correction using llms for hdl design." *DATE*, 2025.

[2] Zhao, Yujie, et al. "PRO-V-R1: Reasoning enhanced programming agent for RTL verification." *DAC* (2026).

How to incorporate Coverage Tool Feedback ?

- Coverage reports are massive!!!
- They also contain **useful textual feedback** signals.

Module Definition

dashboard | hierarchy | modlist | groups | tests | asserts

Module : spi_fsm

SCORE	LINE	COND	TOGGLE	FSM	BRANCH
67.89	68.75	82.35	68.10	55.56	64.71

Source File(s) :
/tmp/cvdp_vcs_spi_partial/src/spi_fsm.v

Module self-instances :

NAME	SCORE	LINE	COND	TOGGLE	FSM	BRANCH
tb_spi_fsm_partial.dut	67.89	68.75	82.35	68.10	55.56	64.71

Module Instance : tb_spi_fsm_partial.dut

Instance :

SCORE	LINE	COND	TOGGLE	FSM	BRANCH
67.89	68.75	82.35	68.10	55.56	64.71

Instance's subtree :

SCORE	LINE	COND	TOGGLE	FSM	BRANCH
67.89	68.75	82.35	68.10	55.56	64.71

Parent :

SCORE	LINE	COND	TOGGLE	FSM	BRANCH NAME
					tb_spi_fsm_partial

Subtrees :

NAME	SCORE	LINE	COND	TOGGLE	FSM	BRANCH
no children						

Since this is the module's only instance, the coverage report is the same as for the module.

Line Cond Toggle FSM Branch

Toggle Coverage for Module : spi_fsm

	Total	Covered	Percent
Totals	17	13	76.47
Total Bits	116	79	68.10
Total Bits 0->1	58	40	68.97
Total Bits 1->0	58	39	67.24
Ports	12	8	66.67
Port Bits	64	27	42.19
Port Bits 0->1	32	14	43.75
Port Bits 1->0	32	13	40.62
Signals	5	5	100.00
Signal Bits	52	52	100.00
Signal Bits 0->1	26	26	100.00
Signal Bits 1->0	26	26	100.00

Port Details

Name	Toggle	Toggle 1: 0->1	Toggle 0: 1->0	Direction
i_clk	Yes	Yes	Yes	INPUT
i_rst_b	No	No	Yes	INPUT
i_data_in[15:0]	No	No	No	INPUT
i_enable	Yes	Yes	Yes	INPUT
i_fault	No	No	No	INPUT
i_clear	No	No	No	INPUT
o_spi_cs_b	Yes	Yes	Yes	OUTPUT
o_spi_clk	Yes	Yes	Yes	OUTPUT
o_spi_data	Yes	Yes	Yes	OUTPUT
o_bits_left[4:0]	Yes	Yes	Yes	OUTPUT
o_done	Yes	Yes	Yes	OUTPUT
o_fsm_state[1:0]	Yes	Yes	Yes	OUTPUT

Signal Details

Module Definition

dashboard | hierarchy | modlist | groups | tests | asserts

Module : spi_fsm

SCORE	LINE	COND	TOGGLE	FSM	BRANCH
67.89	68.75	82.35	68.10	55.56	64.71

Source File(s) :
/tmp/cvdp_vcs_spi_partial/src/spi_fsm.v

Module self-instances :

NAME	SCORE	LINE	COND	TOGGLE	FSM	BRANCH
tb_spi_fsm_partial.dut	67.89	68.75	82.35	68.10	55.56	64.71

Module Instance : tb_spi_fsm_partial.dut

Instance :

SCORE	LINE	COND	TOGGLE	FSM	BRANCH
67.89	68.75	82.35	68.10	55.56	64.71

Instance's subtree :

SCORE	LINE	COND	TOGGLE	FSM	BRANCH
67.89	68.75	82.35	68.10	55.56	64.71

Parent :

SCORE	LINE	COND	TOGGLE	FSM	BRANCH NAME
					tb_spi_fsm_partial

Subtrees :

NAME	SCORE	LINE	COND	TOGGLE	FSM	BRANCH
no children						

Since this is the module's only instance, the coverage report is the same as for the module.

Line Cond Toggle FSM Branch

1 Covered

LINE 87
EXPRESSION (i_enable && ((!i_fault)) && (!i_clear))
-----1-----

-1- -2- -3- Status
0 1 1 Not Covered
1 0 1 Not Covered
1 1 0 Not Covered
1 1 1 Covered

LINE 106
EXPRESSION ((o_bits_left == 5'b0) ? 5'b0 : ((o_bits_left - 5'b1)))
-----1-----

-1- Status
0 Covered
1 Covered

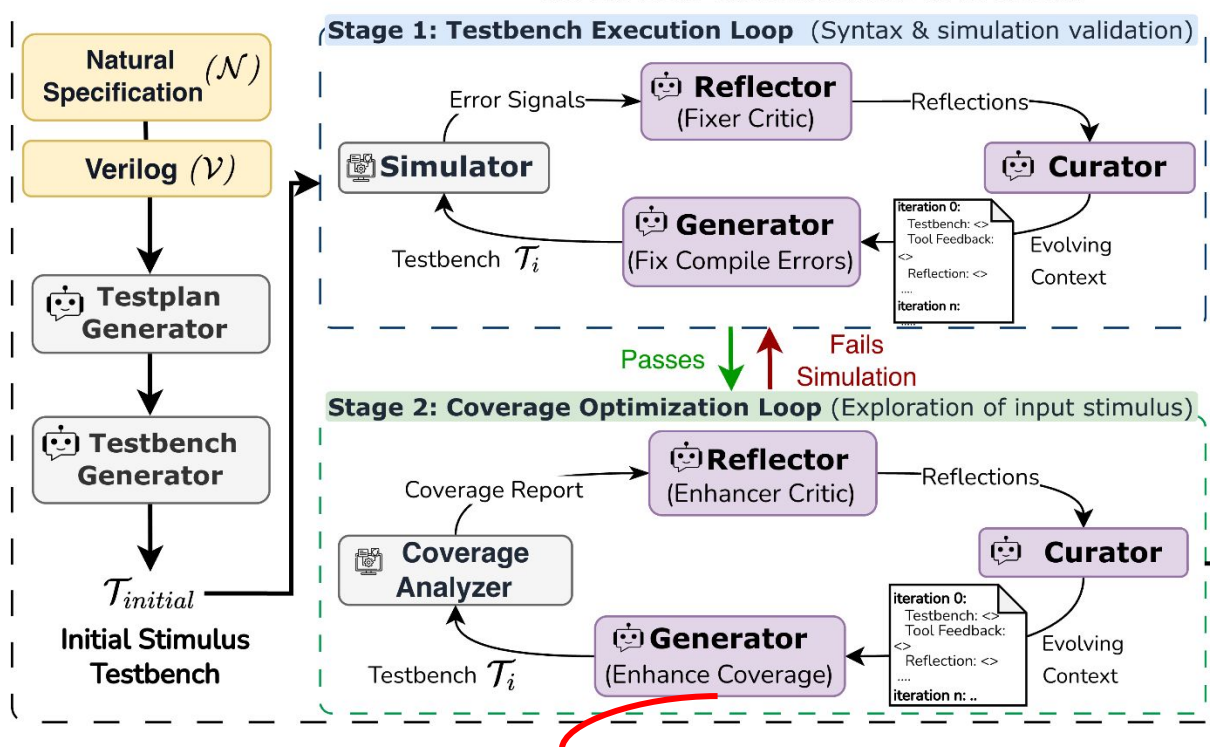
LINE 106
SUB-EXPRESSION (o_bits_left == 5'b0)
-----1-----

-1- Status
0 Covered
1 Covered

LINE 114
EXPRESSION (o_bits_left == 5'b1)
-----1-----

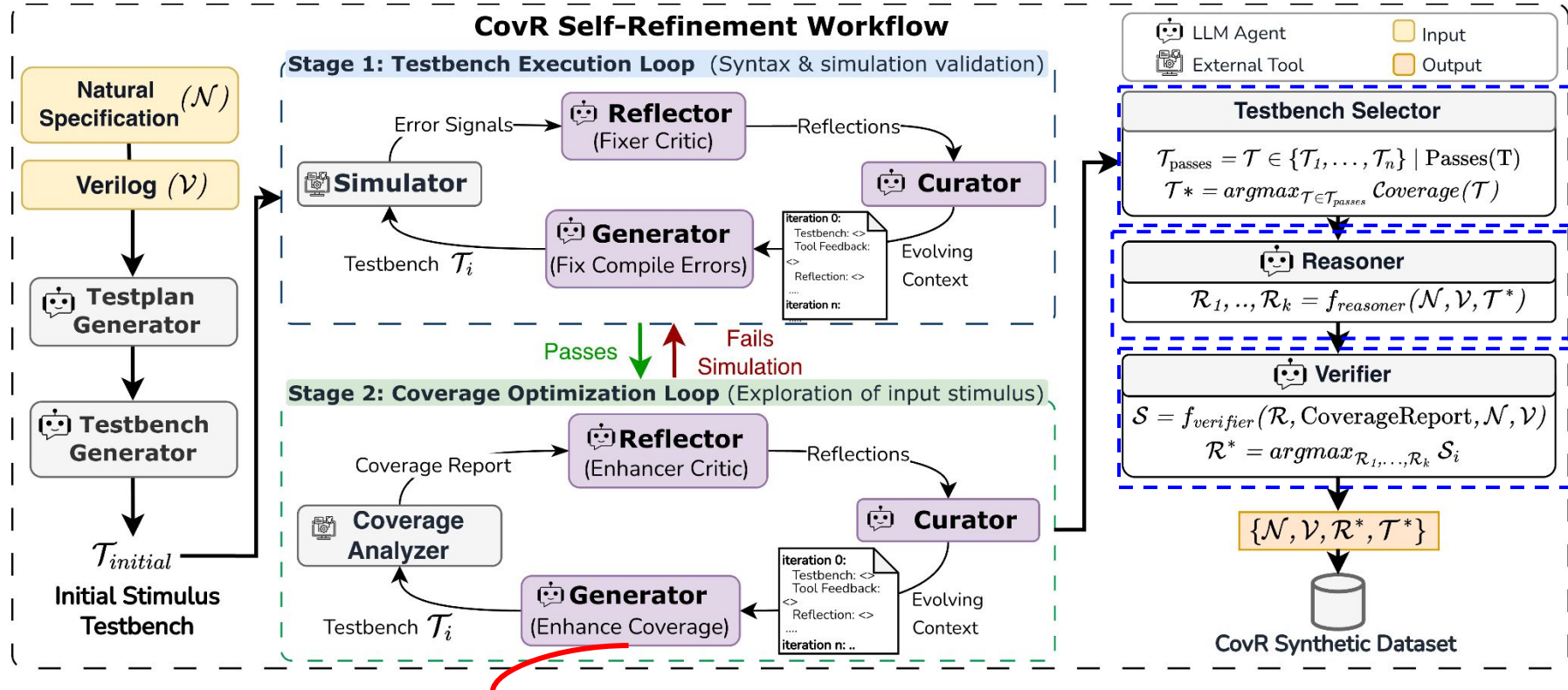
(a) CovR Agentic Workflow

CovR Self-Refinement Workflow



The quality of the generated testbenches on the **Generator**.

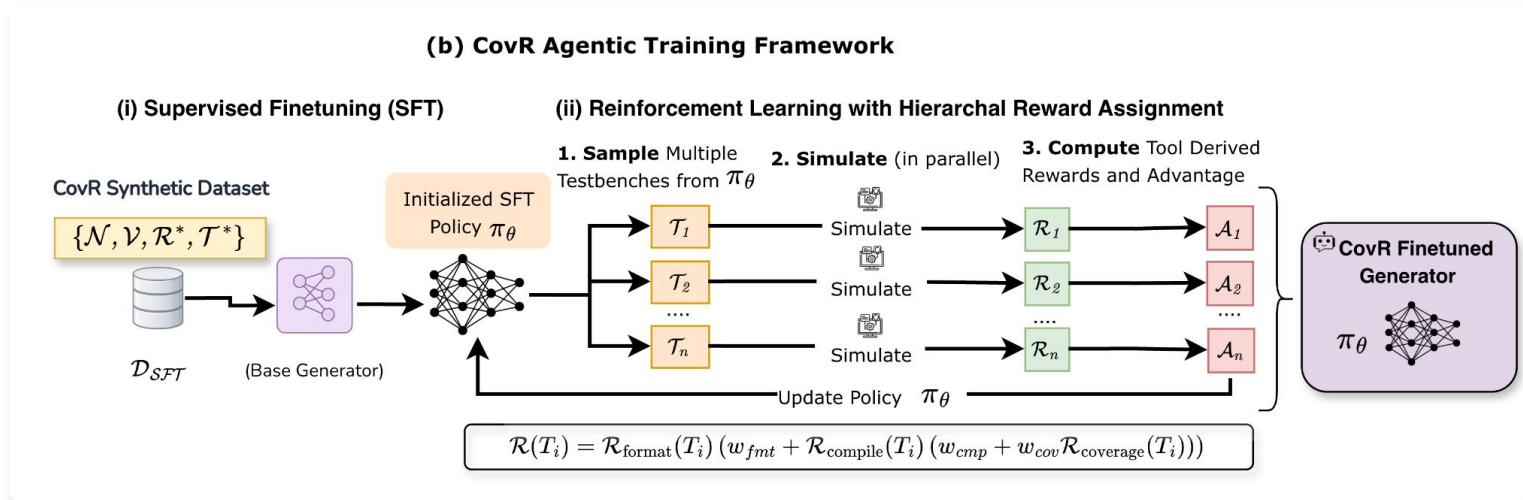
(a) CovR Agentic Workflow



The quality of the generated testbenches on the **Generator**.

(b) CovR Training Framework

- **Two stage training pipeline:**
 - **SFT-Stage:**
 - Learn to generate high coverage testbenches from teacher-generated demonstrations.
 - **RL-Stage:**
 - Refines the SFT policy policy through EDA-tool feedback to maximize code coverage while preserving valid,executable testbenches.



Experimental Setup: Tools & Metrics

Evaluation Set	# Designs
VerilogEval + RTLLMV2.0	203
CVDP (cid012)	83

	Tool
Simulation & Coverage	Synopsys VCS
RTL Mutant Generation	Yosys MCY

Metrics	
pass@k	Testbench compiles
cov@k	Code Coverage

$$q_{i,j} = \begin{cases} \frac{\hat{C}_{i,j}}{C_i^{\max}}, & \text{if the sample runs successfully in simulation} \\ 0, & \text{otherwise.} \end{cases}$$



$$\text{cov@k} = \frac{1}{N} \sum_{i=1}^N \mathbb{E}_{J \subseteq \{1, \dots, n\}, |J|=k} \left[\max_{j \in J} q_{i,j} \right] = \frac{1}{N} \sum_{i=1}^N \sum_{r=k}^n \frac{\binom{r-1}{k-1}}{\binom{n}{k}} q_{i,(r)}$$

Experimental Results

Testbench compiles

✗ Direct Inference
✓ Agentic

Coverage

	Model	Variant	Agentic	VerilogEval + RTLLM V2.0						CVDP (cid012)					
				pass@k (%)			cov@k (%)			pass@k (%)			cov@k (%)		
				k=1	k=5	k=10	k=1	k=5	k=10	k=1	k=5	k=10	k=1	k=5	k=10
Vanilla	gpt-4o-mini	-	✗	88.62	95.25	96.06	81.27	90.01	91.60	71.08	87.15	90.30	62.10	79.78	83.75
	Qwen2.5-Coder-7B-Instruct	-	✗	77.88	93.72	95.07	67.26	86.54	89.54	51.44	75.22	80.72	41.63	65.05	71.49
	Qwen3-4B-Instruct-2507	-	✗	65.02	93.47	96.06	55.56	86.13	90.24	20.00	53.60	69.88	13.82	38.54	51.58
Iterative Feedback	Qwen2.5-Coder-7B-Instruct	-	✓	80.30	90.61	92.12	73.70	84.99	87.08	69.64	82.15	84.34	56.72	70.01	73.01
	Qwen3-4B-Instruct-2507	-	✓	85.22	94.67	96.06	76.34	88.24	90.07	56.87	80.65	85.54	43.32	65.78	71.84
CovR (Ours)	Qwen2.5-Coder-7B-Instruct	-	✓	82.85	96.62	97.04	75.75	92.21	93.51	73.37	93.83	96.39	60.16	83.06	86.62
	Qwen3-4B-Instruct-2507	-	✓	84.78	95.48	96.06	76.90	91.34	92.82	56.75	87.75	96.39	43.00	72.41	81.63

→ **Self-Reflection Loop** improves cov@10 over iterative feedback loop by **6.43%** on VerilogEval+RTLLM V2.0 and **13.61%** on CVDP

→ **SFT+RL+Agentic** progressively improves performance achieving a final **cov@1** of **91.47%** on verilogeval, **79.86%** on CVDP

→ **Qwen-2.5-Coder-CovR** surpasses teacher model (*gpt-4o-mini*) by **7.22%** on VerilogEval and **10.91%** on CVDP on cov@1

Experimental Results

Testbench compiles

✗ Direct Inference
✓ Agentic

Coverage

	Model	Variant	Agentic	VerilogEval + RTLLM V2.0						CVDP (cid012)					
				pass@k (%)			cov@k (%)			pass@k (%)			cov@k (%)		
				k=1	k=5	k=10	k=1	k=5	k=10	k=1	k=5	k=10	k=1	k=5	k=10
Vanilla	gpt-4o-mini	-	✗	88.62	95.25	96.06	81.27	90.01	91.60	71.08	87.15	90.30	62.10	79.78	83.75
	Qwen2.5-Coder-7B-Instruct	-	✗	77.88	93.72	95.07	67.26	86.54	89.54	51.44	75.22	80.72	41.63	65.05	71.49
	Qwen3-4B-Instruct-2507	-	✗	65.02	93.47	96.06	55.56	86.13	90.24	20.00	53.60	69.88	13.82	38.54	51.58
Iterative Feedback	Qwen2.5-Coder-7B-Instruct	-	✓	80.30	90.61	92.12	73.70	84.99	87.08	69.64	82.15	84.34	56.72	70.01	73.01
	Qwen3-4B-Instruct-2507	-	✓	85.22	94.67	96.06	76.34	88.24	90.07	56.87	80.65	85.54	43.32	65.78	71.84
CovR (Ours)	Qwen2.5-Coder-7B-Instruct	-	✓	82.85	96.62	97.04	75.75	92.21	93.51	73.37	93.83	96.39	60.16	83.06	86.62
	Qwen3-4B-Instruct-2507	-	✓	84.78	95.48	96.06	76.90	91.34	92.82	56.75	87.75	96.39	43.00	72.41	81.63
	CovR-Qwen3-4B-Instruct-2507	SFT	✗	78.91	88.41	90.15	73.53	83.56	85.65	56.75	84.62	87.95	50.78	76.98	80.85
		SFT-W/Reason	✗	77.93	93.83	<u>96.55</u>	71.34	88.49	92.07	65.42	90.98	<u>93.98</u>	56.45	82.05	85.81
		SFT-W/Reason+RL	✗	90.79	96.70	97.04	86.50	93.19	93.81	<u>78.92</u>	<u>91.41</u>	<u>92.77</u>	71.46	85.94	87.76
		SFT-W/Reason+RL	✓	95.71	97.04	97.04	91.47	<u>94.03</u>	<u>94.27</u>	88.19	93.94	95.18	79.86	89.66	91.39
	CovR-Qwen-2.5-coder-7b	SFT	✗	77.00	89.80	92.10	72.00	85.20	87.50	72.17	91.75	92.77	64.69	85.27	87.24
		SFT-W/Reason	✗	84.50	<u>96.51</u>	97.04	78.34	91.67	<u>92.90</u>	69.04	92.24	93.98	61.12	84.21	86.79
		SFT-W/Reason+RL	✗	<u>92.51</u>	96.50	<u>96.55</u>	<u>88.49</u>	<u>93.29</u>	<u>93.61</u>	80.48	90.83	91.56	73.01	85.73	87.25
		SFT-W/Reason+RL	✓	<u>94.38</u>	<u>97.03</u>	97.04	<u>90.38</u>	94.10	94.30	<u>87.10</u>	<u>94.89</u>	<u>95.18</u>	<u>78.72</u>	<u>89.40</u>	<u>90.44</u>

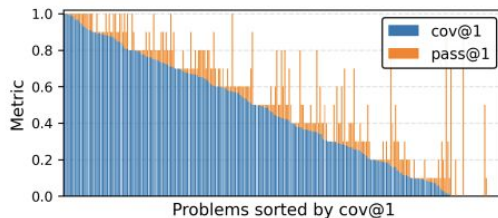
→ **Self-Reflection Loop** improves cov@10 over iterative feedback loop by **6.43%** on VerilogEval+RTLLM V2.0 and **13.61%** on CVDP

→ **SFT+RL+Agentic** progressively improves performance achieving a final **cov@1** of **91.47%** on verilogeval, **79.86%** on CVDP

→ **Qwen-2.5-Coder-CovR** surpasses teacher model (*gpt-4o-mini*) by **7.22%** on VerilogEval and **10.91%** on CVDP on cov@1

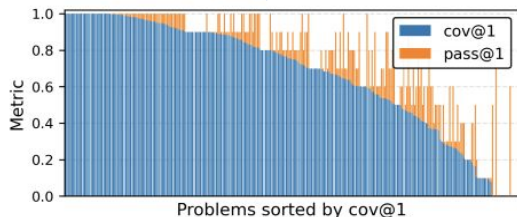
Experimental Results: cov@1 progression

1. Vanilla



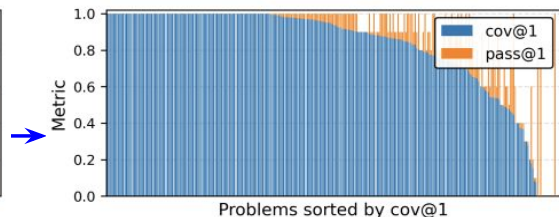
(a) Vanilla

2. +SFT



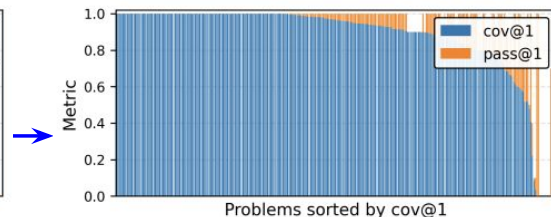
(b) CovR-SFT-W/Reason

3. +RL



(c) CovR-SFT-W/Reason+RL

4. +Agentic



(d) CovR-SFT-W/Reason+RL (Agentic)

→ **SFT + RL + Agentic** refinement progressively shifts generated stimuli toward near-complete RTL coverage.

Experimental Results: cov@1 progression

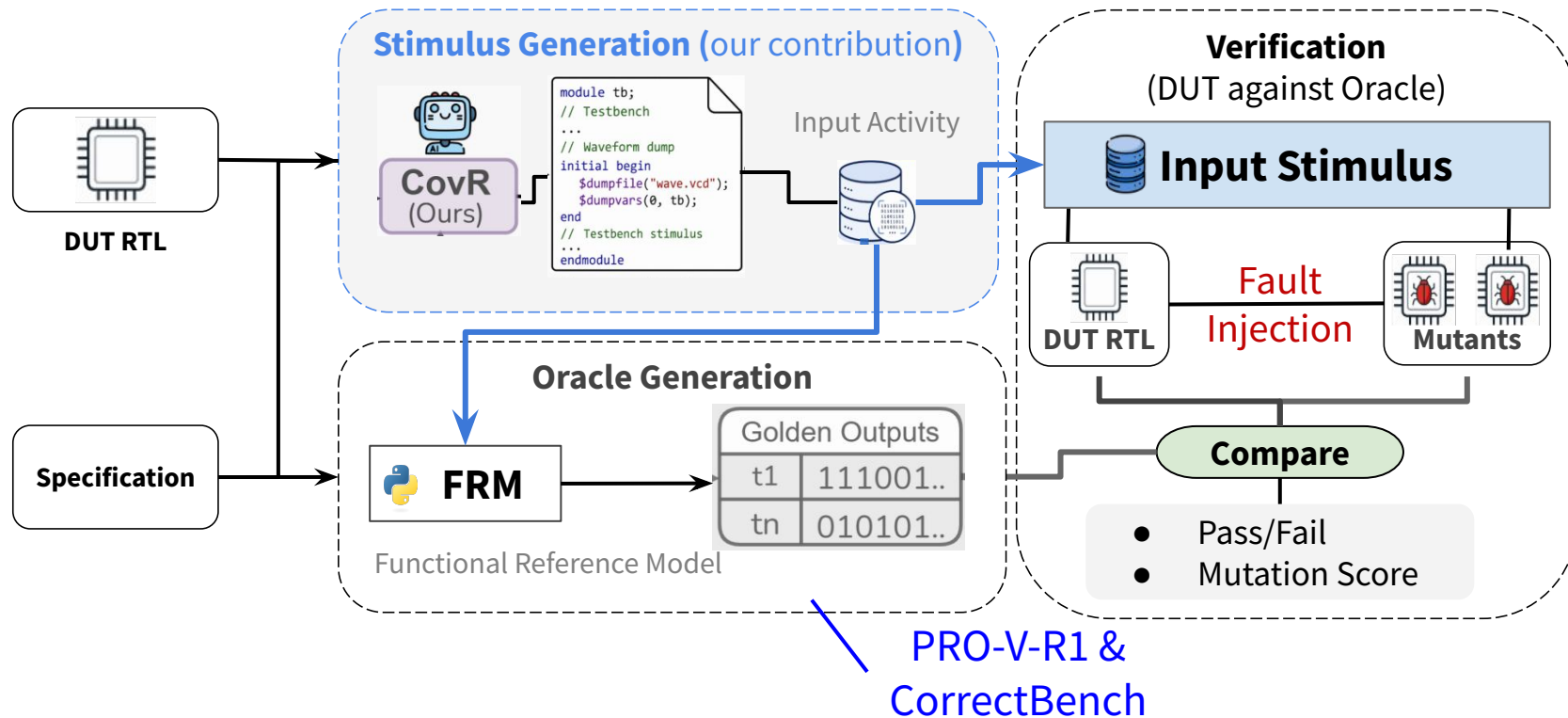


FSM is the hardest

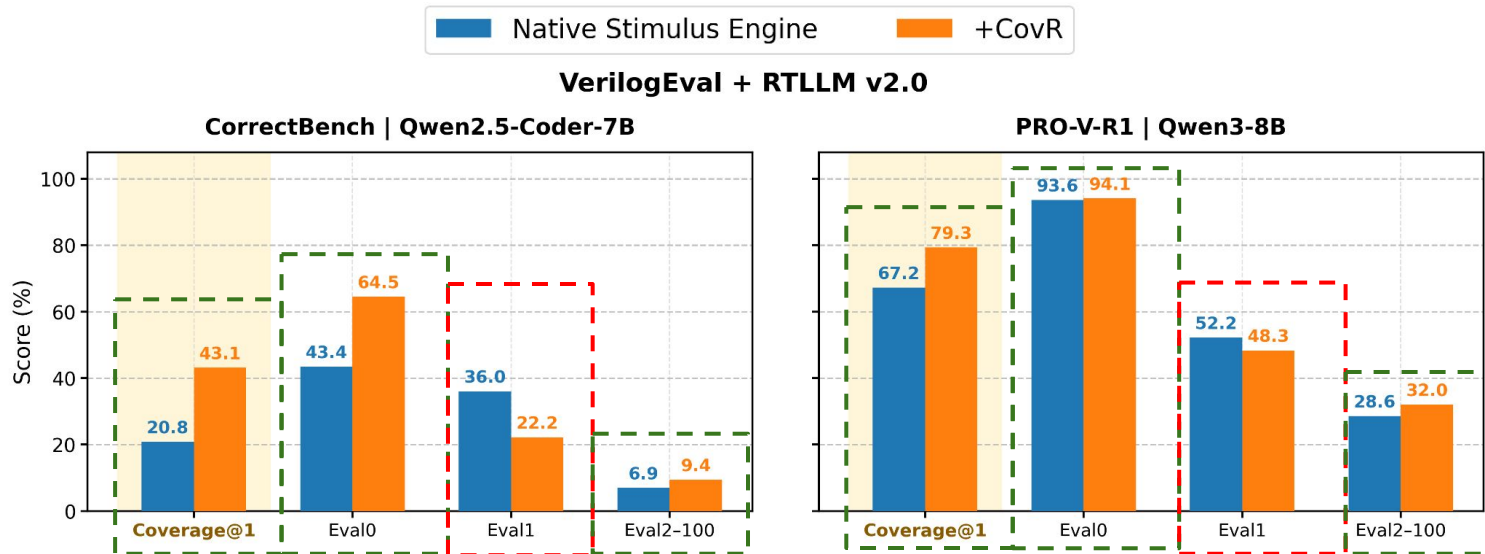
→ SFT provides a strong coverage boost, while RL and agentic feedback progressively close the remaining gap (**44% → 64% → 79% → 86%**).

CovR as a plug-in Stimulus Engine

CovR enables the driver (stimulus generation) track.



Integrating CovR With Full Verification Workflows



- **Coverage** increased by 17.2% on average
- **Eval0**: Verification harness compiles without syntax or runtime errors increased by 10.8% on average
- **Eval1**: Generated reference model (FRM) correctly accepts the golden RTL decreased by 8.9%
- **Eval2-100**: Detects 100% of the RTL mutants increased by 3.0%.

Case Study: CovR Reveals Latent FRM Bugs

(a) Specification

TopModule implements a level-sensitive D latch using an always block. When ena=1, the output follows d. When ena=0, the output holds its previous value:

$$q^+ = \begin{cases} d, & \text{ena} = 1 \\ q, & \text{ena} = 0 \end{cases}$$

(b) Benchmark RTL

```
module TopModule (  
  input d,  
  input ena,  
  output logic q  
);  
  
always@(*) begin  
  if (ena)  
    q = d;  
end  
  
endmodule
```

Golden RTL
in VerilogEval

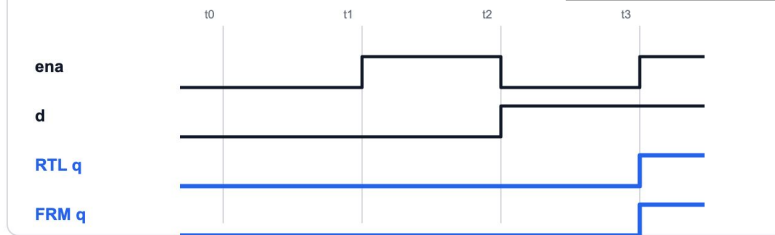
(c) Generated Pro-V FRM

```
class GoldenDUT:  
  def __init__(self):  
    self.q = 0  
    self.prev_ena = 0  
  
  def load(self, inputs):  
    d = int(inputs["d"], 2)  
    ena = int(inputs["ena"], 2)  
    if ena == 1 and self.prev_ena == 0: BUG  
      self.q = d  
      self.prev_ena = ena  
  
  return {"q": format(self.q & 0x1, "01b")}
```

Edge Triggered

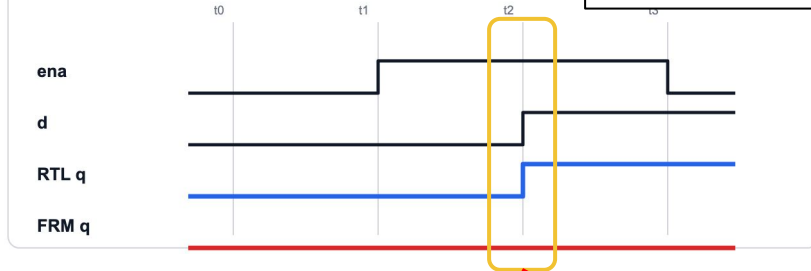
Bug: The FRM incorrectly models ena as edge-triggered. A D latch should update whenever ena=1.

(d) Pro-V Stimulus



75%
Coverage

(e) CovR Stimulus



100%
Coverage

FRM isn't transparent when ena is high!

Case Study: CovR Improves Mutation Kill Score

(a) Specification

TopModule decodes four keyboard scancodes into one-hot direction signals:

E06B → left, E072 → down, E074 → right, E075 → up.

All other scancodes should produce left=down=right=up=0.



(b) Full Benchmark RTL

```
module TopModule (  
  input [15:0] scancode,  
  output reg left,  
  output reg down,  
  output reg right,  
  output reg up  
);  
  
always @(*) begin  
  {up, left, down, right} = 0;  
  
  case (scancode)  
    16'h06b: left = 1;  
    16'h072: down = 1;  
    16'h074: right = 1;  
    16'h075: up = 1;  
  endcase  
end  
  
endmodule
```

(c) Representative Mutant Faults

Inverted decode predicates

M0 assign right = ~(scancode == 16'h074);

M2 assign up = ~(scancode == 16'h075);

M4 assign down = ~(scancode == 16'h072);

M6 assign left = ~(scancode == 16'h06b);

Stuck-at-high output faults

M1 assign left = 1'h1;

M5 assign up = 1'h1;

M8 assign right = 1'h1;

Stuck-at-low output faults

M3 assign down = 1'h0;

M7 assign left = 1'h0;

M9 assign right = 1'h0;

(d) Pro-V stimulus

Mutation score: 70.0% - Killed: 7/10

test	0	1	2	3	4
scancode	1f7c	f072	29f1	30e1	fb17
left	0	0	0	0	0
down					
right					
up					

Killed: 7/10

Survived: M3, M7, M9

(e) CovR stimulus

Mutation score: 100.0% - Killed: 10/10

test	0	1	2	3
scancode	e06b	e072	e074	e075
left	1	0	0	0
down	0	1	0	0
right	0	0	1	0
up	0	0	0	1

Killed: 10/10

No survivors

Thank You!

<https://github.com/scale-lab/CovR>

